Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

The dataset was loaded through Google Drive on Colab

1.1.1. Sample the data and combine the files.

Once the folder containing the dataset was located in the drive, we created a dataframe to store the sampled data for the whole year. Once the file processing starts, we store the data in the dataframe after checking if we are using the correct date and time column. We then loop through each day in the file. Within this process, we loop through each hour of the day, and during this, we sample 5%(as mentioned in the assignment) of the data from each hour and append the data frame. After the entire loop is executed, we save the sampled data in the drive titled “sample_trip_data”. This dataset has 1,896,400 entries, which is more than the suggested 250k to 300k entries, suggesting sampling 1% of the data to get the entries close to the suggested values.

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

We have fixed the index by starting the indexing with zero

2.1.2. Combine the two airport_fee columns

We have combined the two airport fee columns by adding them to each other or by replacing the NaN, and by doing this, we aren’t missing any extra charges that have been placed in the airport, like wait time.

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

Listed is the percentage of missing values in each column

Proportion of missing values per column: Congestion_surcharge: 3.42, passenger_count: 3.42, store_and_fwd_flag: 3.42, RatecodeID: 3.42

As there are very few missing values present in the dataset, we do not need to drop any columns for this dataset; we only need to check what

type of data is missing and replace the value with the mode or median or remove the row if needed.

2.2.2. Handling missing values in passenger_count

In case of passenger_count, we have replaced the missing data with the median and removed all the data where the count was equal to 0, as this is not realistic. We have chosen median over mode in order to make sure the data is not biased, as there are a very large number of solo riders already present in the dataset.

2.2.3. Handle missing values in RatecodeID.

The missing rate code values were replaced with mode.

2.2.4. Impute NaN in congestion_surcharge

As mentioned, we have imputed NaN in places where the congestion charge is not added.

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

In the payment type section, we have removed data related to invalid payment types. We have removed the top and bottom 10% of the data from the trip distance and tip amount columns to remove the extreme outlier cases. We have also removed trip distances that exceed 250 miles.

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

categorical_vars = ['VendorID', 'RatecodeID', 'PULocationID', 'DOLocationID', 'payment_type']

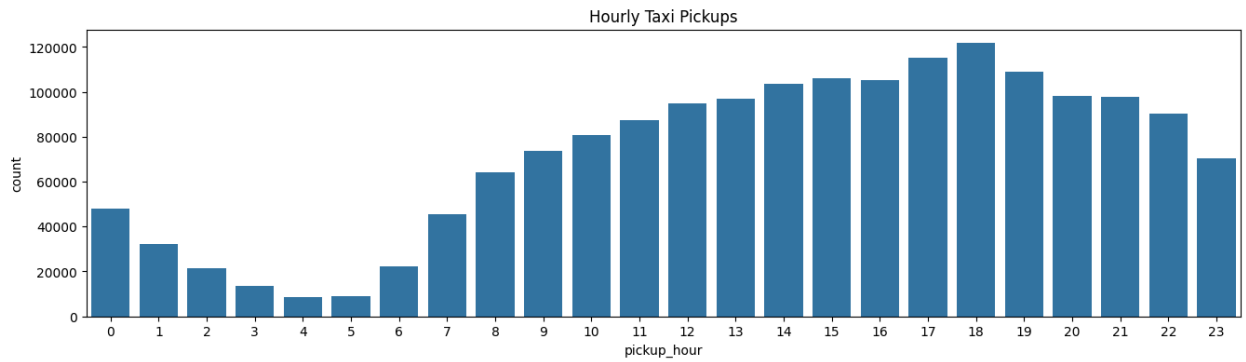
temporal_vars = ['tpep_pickup_datetime', 'tpep_dropoff_datetime', 'pickup_hour']

numerical_vars = ['passenger_count', 'trip_distance', 'trip_duration']

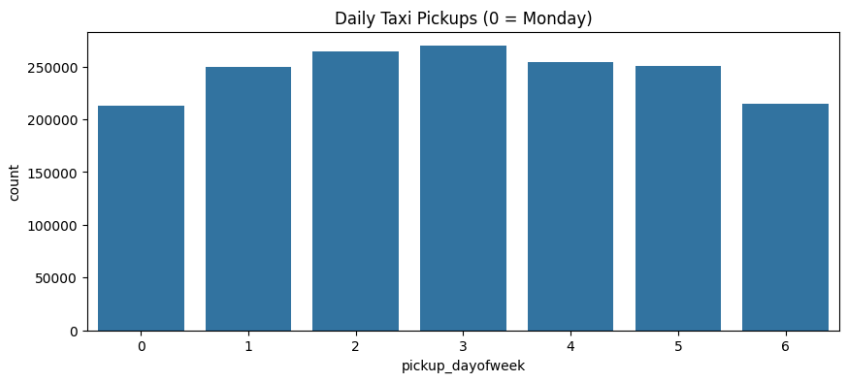
monetary_vars = ['fare_amount', 'extra', 'mta_tax', 'tip_amount', 'tolls_amount', 'improvement_surcharge', 'total_amount', 'congestion_surcharge', 'airport_fee']

3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

Attached below are the graphs of the required data

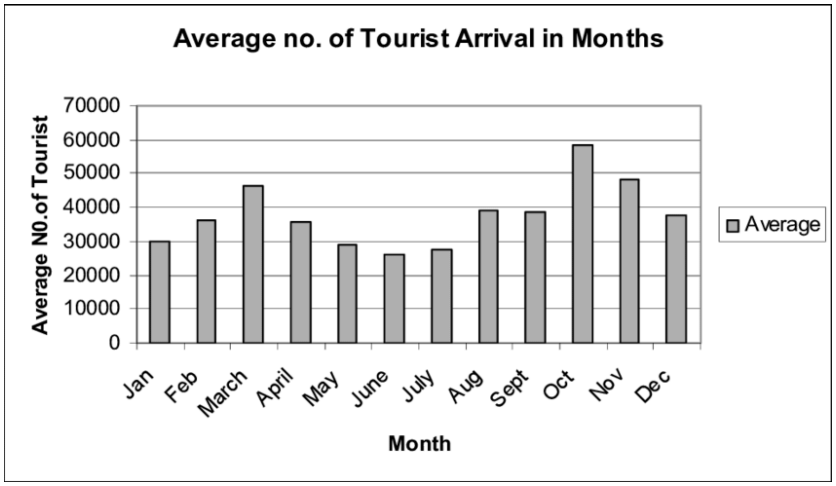


As seen from the graph above, there is a rapid rise in pickups starting at 6 AM, reaching its peak at 6 PM. This shows the peak hours of the day, and it also lines up with the average workday.



Above is the graph of the daily taxi pickups per day of the week; the data shows a normal distribution, but given the context, there are not many reasons for this to be the case. We can only infer that there are more people in transit on Thursday compared to any other day of the week, such as those with compulsory work from the office in hybrid settings.

Above are the monthly taxi pickups. There are a few months with higher pickups than others, and with further research, these months are in line with the tourist arrivals months. The graph is located below.



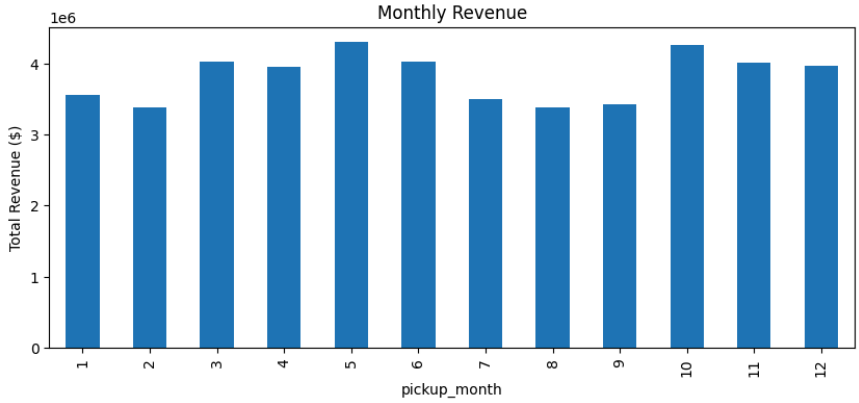
This trend is also visible in hotel pricing in the city, and we can correlate the tourists with the large number of pick-ups in the following months.

3.1.3. Filter out the zero/negative values in fares, distance and tips

Negative/Zero Values in Key Financial Columns (after cleaning):

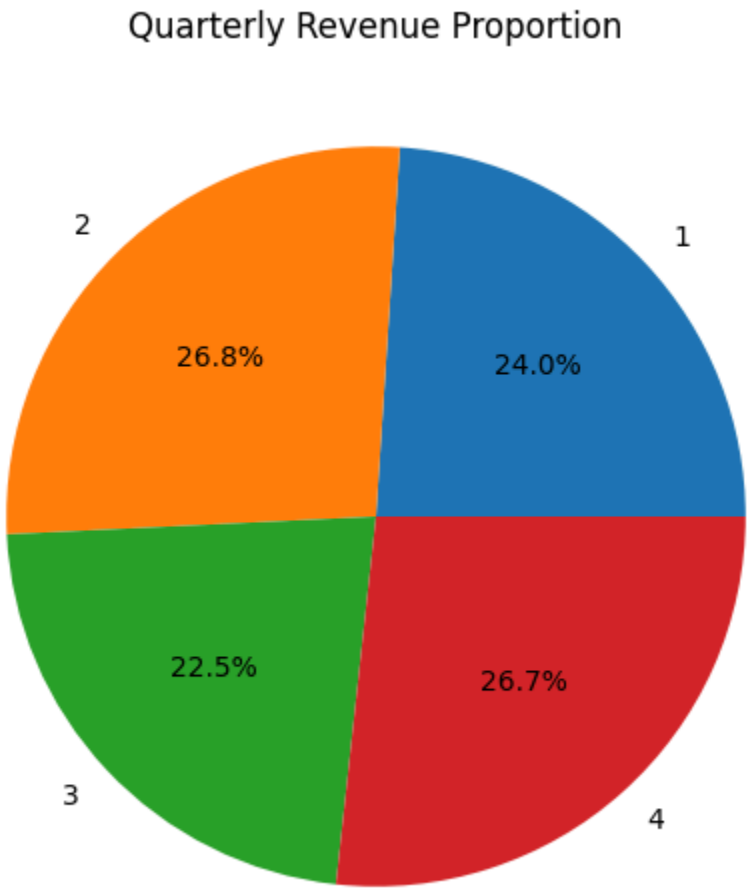
fare_amount: 0 zero/negative values
tip_amount: 370821 zero/negative values
total_amount: 0 zero/negative values
trip_distance: 0 zero/negative values

3.1.4. Analyse the monthly revenue trends



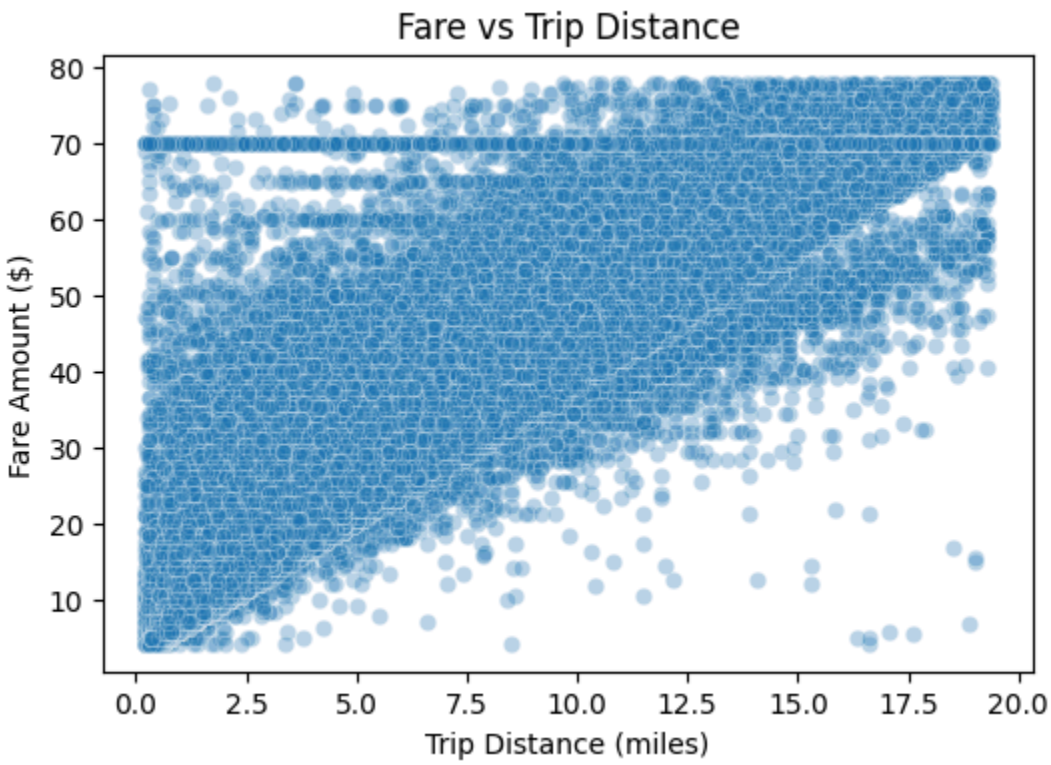
From the above graph, we can infer that the revenue is higher during the months with a larger number of tourists. However, the difference is very small, so we can't say for certain the overall dependence of taxi services on tourism.

3.1.5. Find the proportion of each quarter's revenue in the yearly revenue



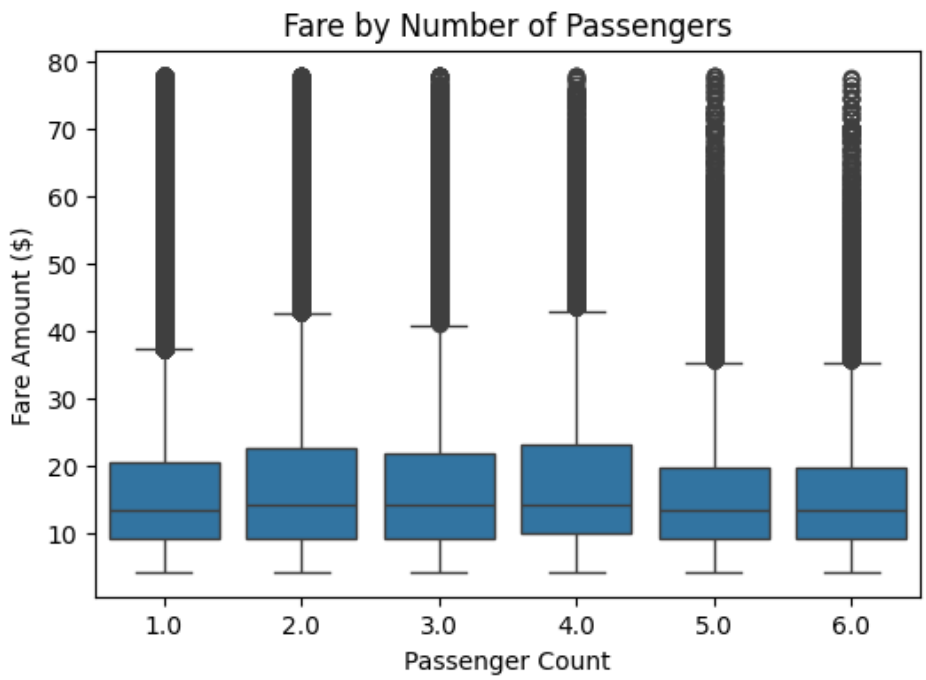
From the above pie chart, we can see that the quarters with the most tourists have a slight increase in revenue compared to the other quarters, providing us with a better insight into the effects of tourism on the taxi industry.

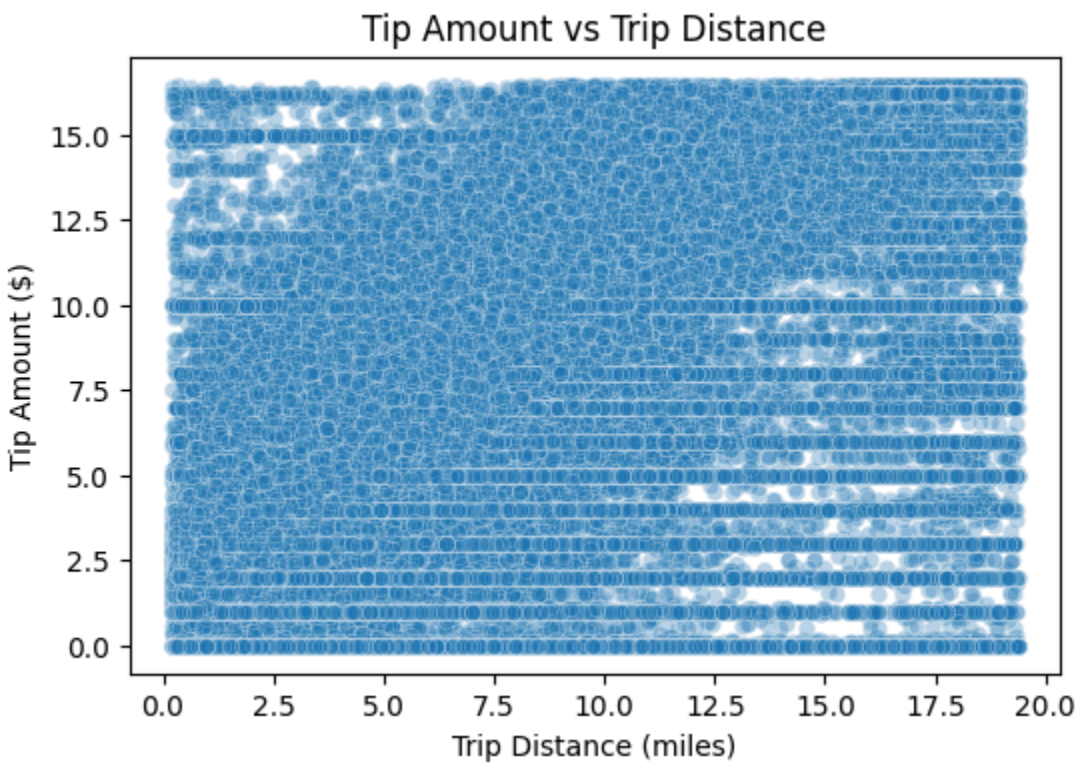
3.1.6. Analyse and visualise the relationship between distance and fare amount



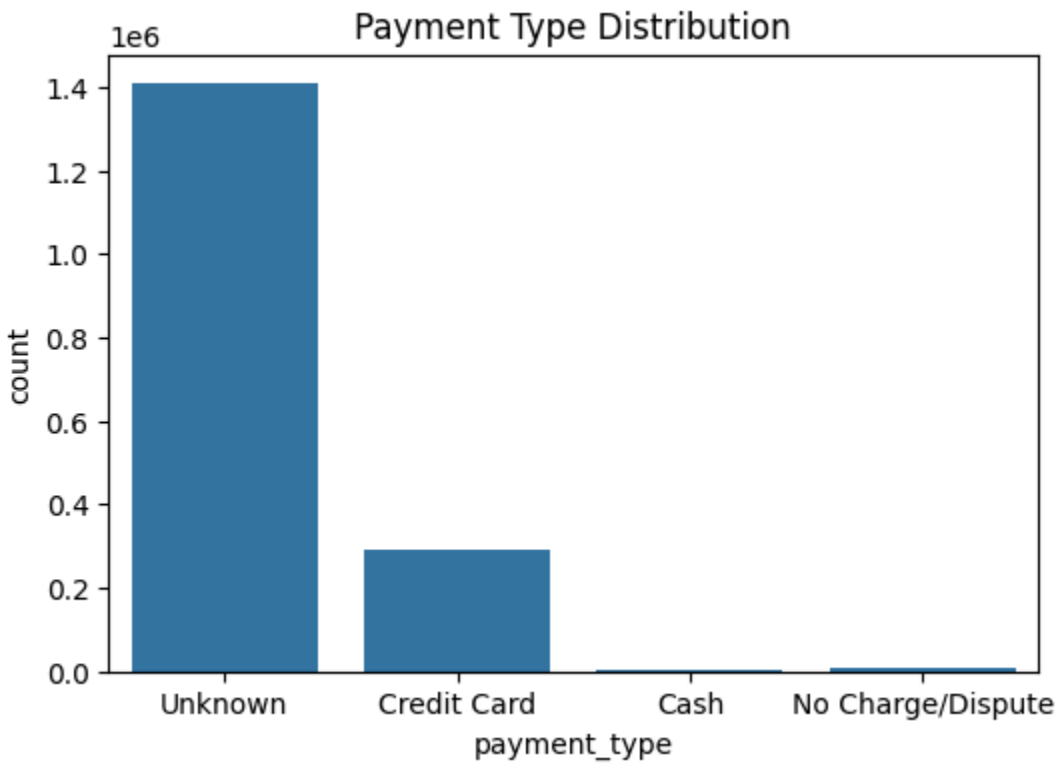
From the above graph, we can infer that the fare amount is not directly proportional to the trip distance. As many other factors, such as time, place and day, cause the fare amount to vary by a large margin even within the same trip distance, but there seems to be a trend with 70$ in case for all trip distances, the large fare for short distances can be due to traffic wait charges. And while looking on the web, $ 70 is a capped rate for taxis to and from JFK International Airport from Manhattan.

3.1.7. Analyse the relationship between fare/tips and trips/passengers





3.1.8. Analyse the distribution of different payment types



3.1.9. Load the taxi zones shapefile and display it

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	OBJECTID	263 non-null	int32
1	Shape_Leng	263 non-null	float64
2	Shape_Area	263 non-null	float64
3	zone	263 non-null	object
4	LocationID	263 non-null	int32
5	borough	263 non-null	object
6	geometry	263 non-null	geometry

3.1.10. Merge the zone data with trips data

```
trip_counts =
df_cleaned.groupby('PULocationID').size().reset_index(name='trip_count')
trip_counts.head()
```

```
#Merge Zones with Trip Data
zones_trips = zones.merge(trip_counts, how='left',left_on='LocationID',
right_on='PULocationID')
```

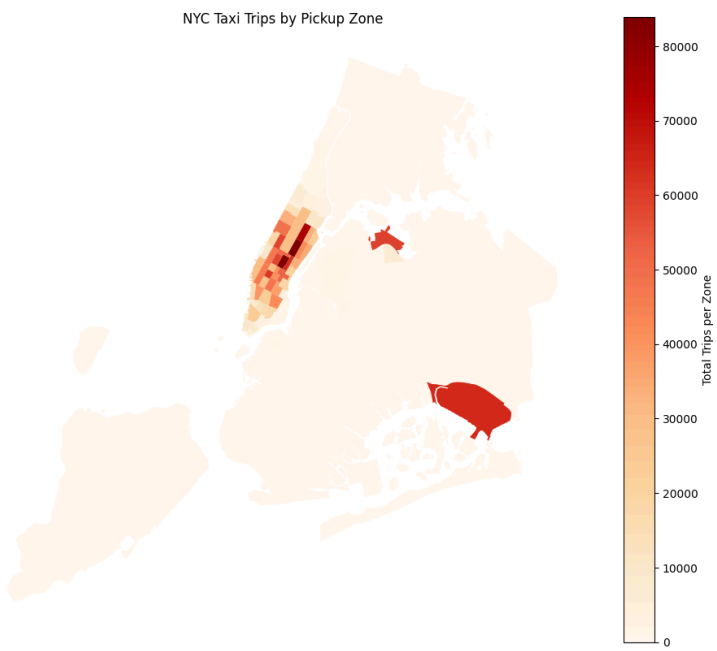
3.1.11. Find the number of trips for each zone/location ID

LocationID		zone	borough	trip_count
236	237	Upper East Side South	Manhattan	83970
160	161	Midtown Center	Manhattan	82414
235	236	Upper East Side North	Manhattan	74861
131	132	JFK Airport	Queens	63858
161	162	Midtown East	Manhattan	63105
185	186	Penn Station/Madison Sq West	Manhattan	61033
137	138	LaGuardia Airport	Queens	59196
141	142	Lincoln Square East	Manhattan	58684
229	230	Times Sq/Theatre District	Manhattan	57840
169	170	Murray Hill	Manhattan	52198

3.1.12. Add the number of trips for each zone to the zones dataframe

```
zones_trips = zones.merge(trip_counts, how='left', left_on='LocationID',
right_on='PULocationID')
```

3.1.13. Plot a map of the zones showing number of trips



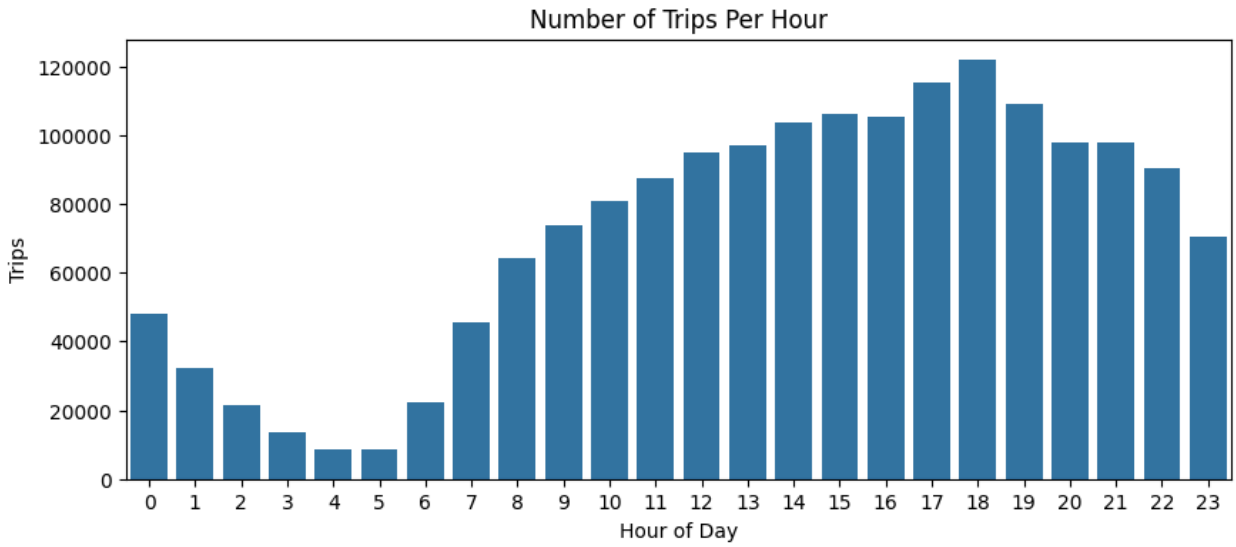
3.1.14. Conclude with results

3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

```
#Ensure datetime columns

df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'])

df['tpep_dropoff_datetime'] = pd.to_datetime(df['tpep_dropoff_datetime'])


#Create time-related features

df['pickup_hour'] = df['tpep_pickup_datetime'].dt.hour

df['pickup_day'] = df['tpep_pickup_datetime'].dt.dayofweek

df['trip_duration_mins'] = (df['tpep_dropoff_datetime'] -
df['tpep_pickup_datetime']).dt.total_seconds() / 60


#Identify slowest routes by hour

avg_duration = df.groupby(['PULocationID', 'DOLocationID',
'pickup_hour'])['trip_duration_mins'].mean().reset_index()

avg_distance = df.groupby(['PULocationID',
'DOLocationID'])['trip_distance'].mean().reset_index()


#Merge to calculate average speed = distance / duration

route_speed = pd.merge(avg_duration, avg_distance,
on=['PULocationID', 'DOLocationID'])

route_speed['avg_speed_mph'] = route_speed['trip_distance'] /
(route_speed['trip_duration_mins'] / 60)

slowest_routes =
route_speed.sort_values(by='avg_speed_mph').head(10)
```

3.2.2. Calculate the hourly number of trips and identify the busy hours



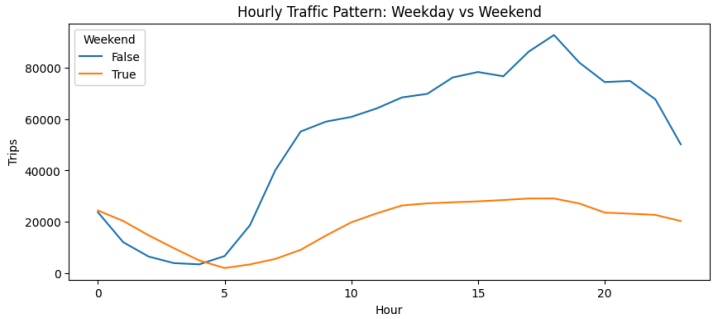
Busiest Hour: 18 with 121738 trips

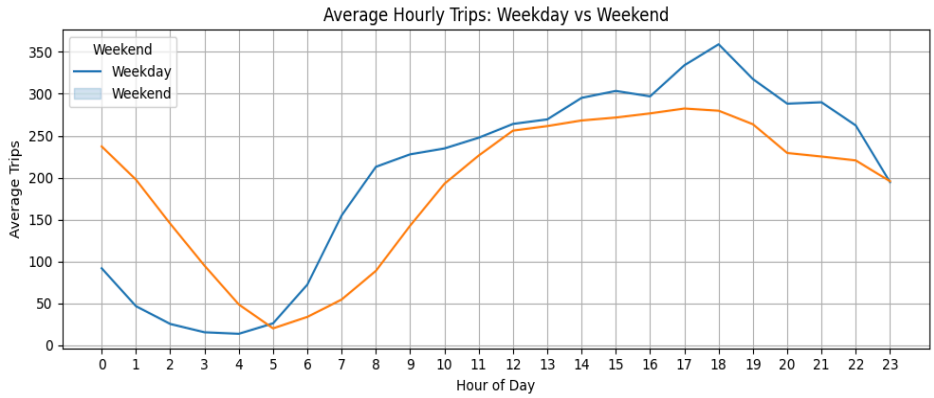
3.2.3. Scale up the number of trips from above to find the actual number of trips

Scaled trip counts for 5 busiest hours:

pickup_hour	Trips
18	2434760.0
17	2305620.0
19	2181660.0
15	2125000.0
16	2101920.0

3.2.4. Compare hourly traffic on weekdays and weekends





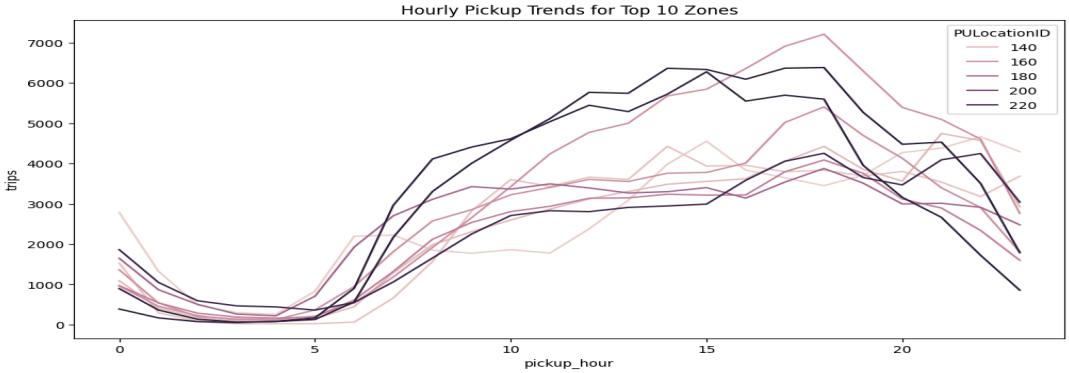
3.2.5. Identify the top 10 zones with high hourly pickups and drops

Top 10 Pickup Zones:

PULocationID	Pickups
237	83976
161	82422
236	74870
132	63888
162	63110
186	61038
138	59207
142	58685
230	57848
170	52203

Top 10 Dropoff Zones:

DOLocationID	Dropoffs
236	78180
237	74637
161	68421
230	52824
170	52125
162	50037
142	49314
239	48522
141	46541
68	44241



3.2.6. Find the ratio of pickups and dropoffs in each zone

Top 10 Pickup/Dropoff Ratios:

	pickup_count	dropoff_count	ratio
44	2.0	0.0	200000.000000
199	2.0	0.0	200000.000000
84	1.0	0.0	100000.000000
70	7532.0	761.0	9.897503
132	63888.0	12647.0	5.051633
138	59207.0	19957.0	2.966728
204	2.0	1.0	1.999980
186	61038.0	38392.0	1.589862
114	23382.0	16861.0	1.386750
43	29613.0	21492.0	1.377862

Bottom 10 Pickup/Dropoff Ratios:

	pickup_count	dropoff_count	ratio
99	0.0	1.0	0.0
109	0.0	2.0	0.0
187	0.0	3.0	0.0
221	0.0	7.0	0.0
30	0.0	18.0	0.0
184	0.0	17.0	0.0
118	0.0	11.0	0.0
245	0.0	6.0	0.0
27	0.0	24.0	0.0
251	0.0	5.0	0.0

3.2.7. Identify the top zones with high traffic during night hours

Top Night Pickup Zones:

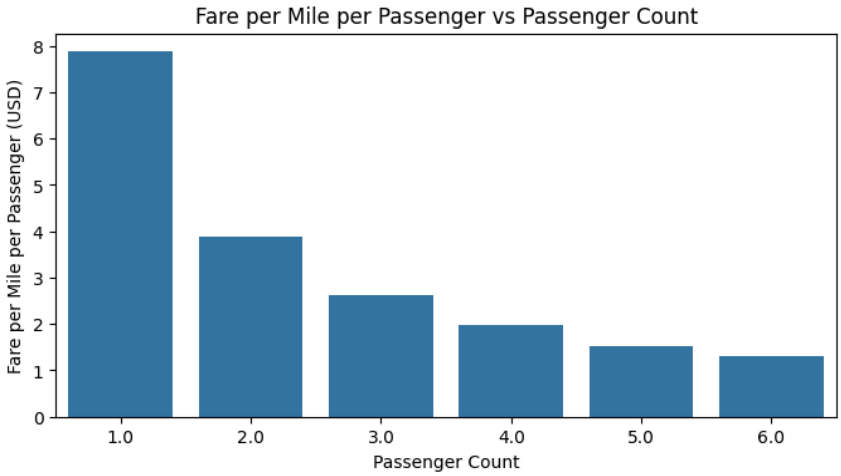
PULocationID	Pickups
79	15052
249	12122
132	10272
48	10098
148	9347
114	8510
230	7811
186	6682
164	5900
68	5785

3.2.8. Find the revenue share for nighttime and daytime hours

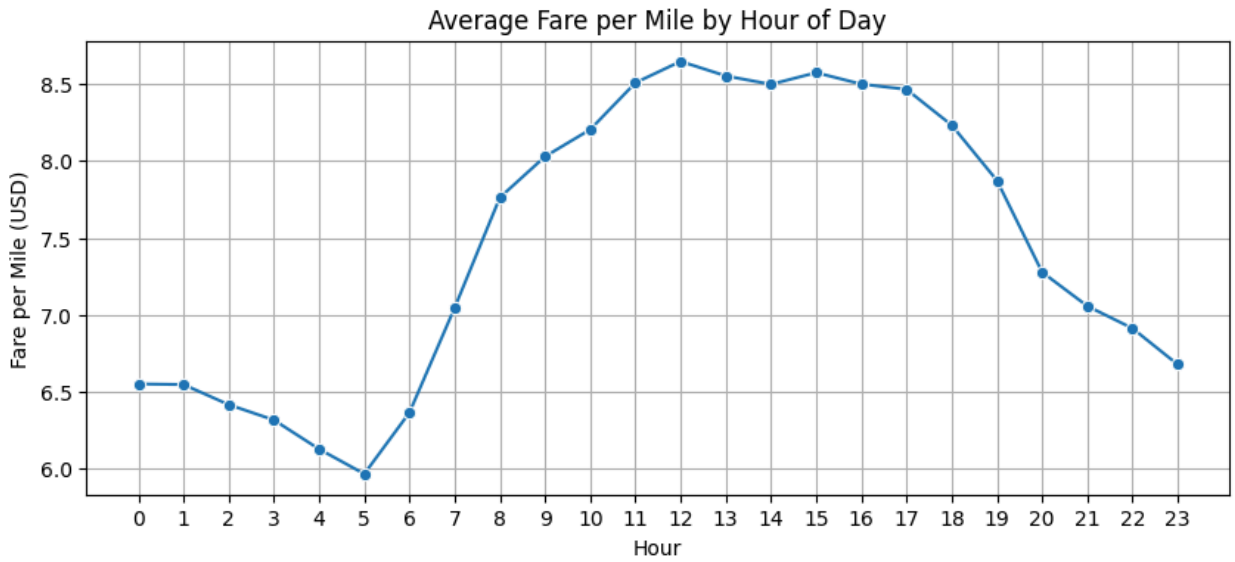
Nighttime Revenue Share: 12.08%

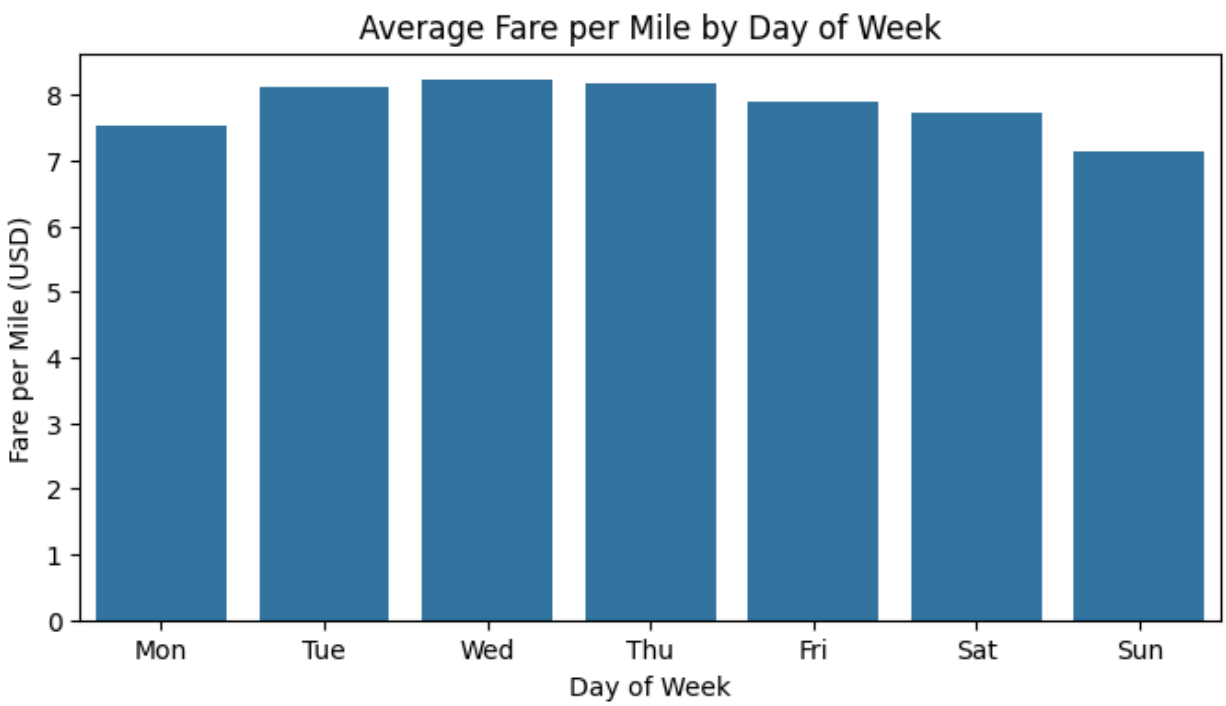
Daytime Revenue Share: 87.92%

3.2.9. For the different passenger counts, find the average fare per mile per passenger

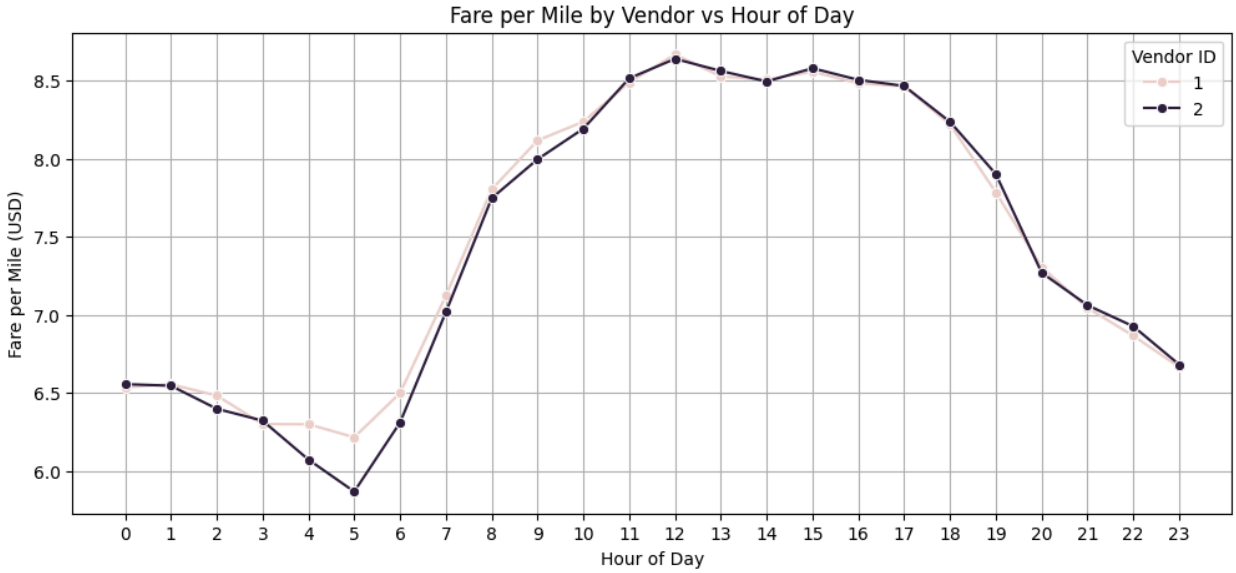


3.2.10. Find the average fare per mile by hours of the day and by days of the week

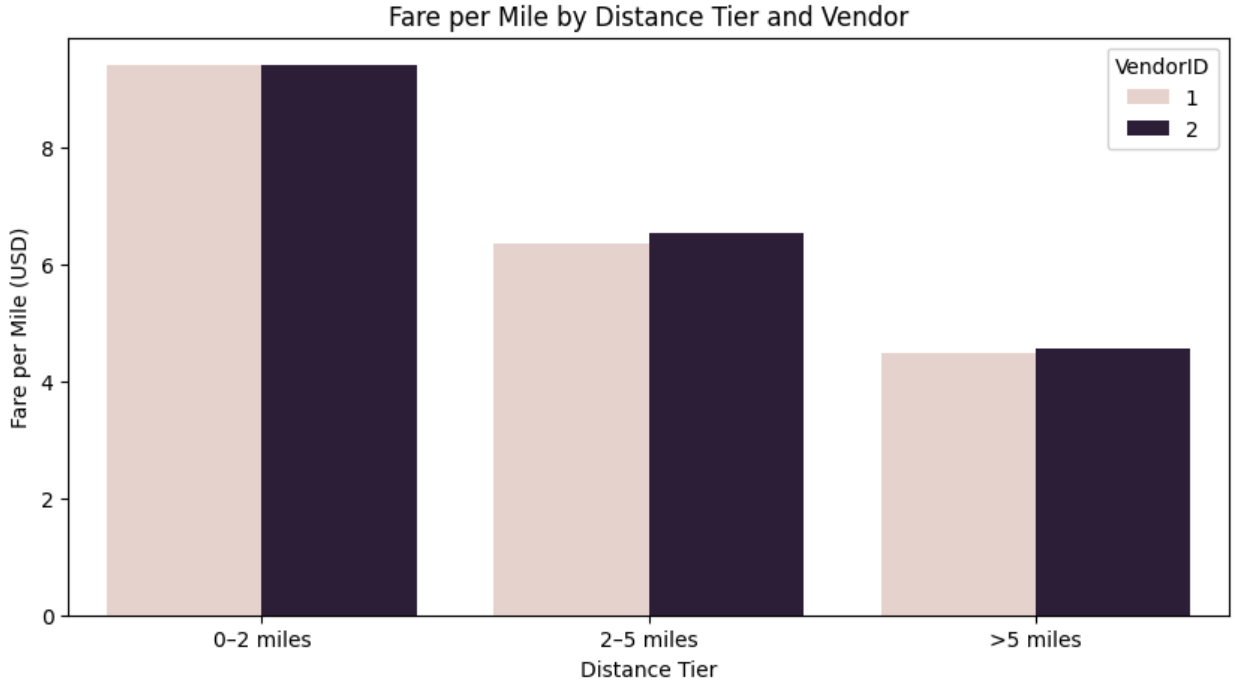




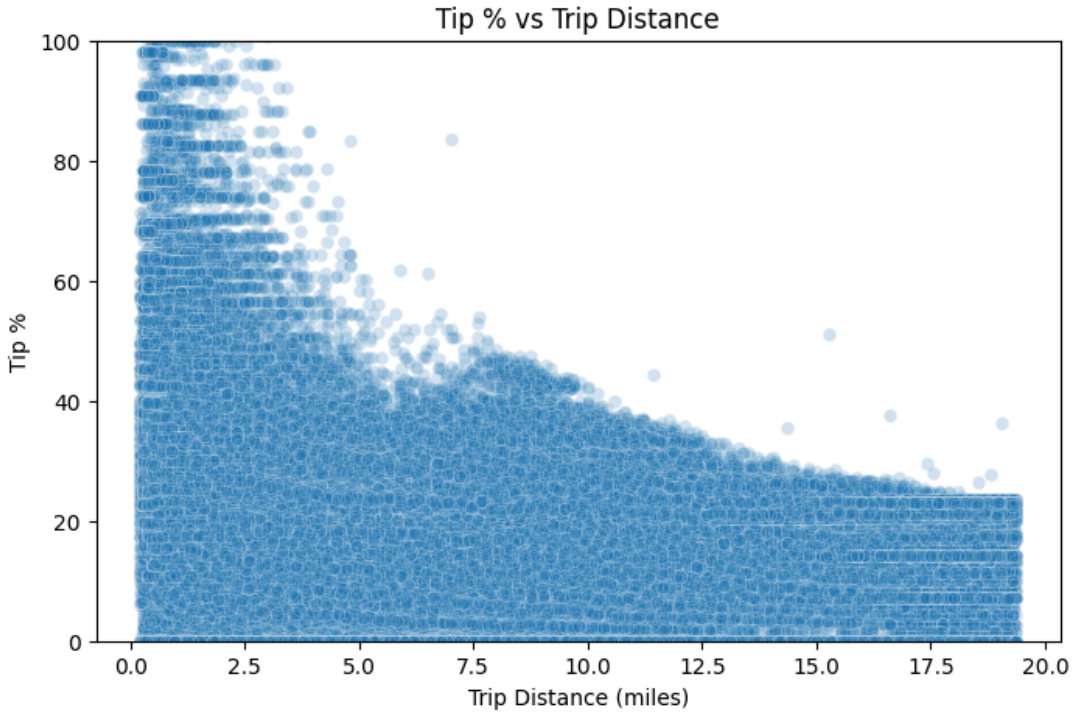
3.2.11. Analyse the average fare per mile for the different vendors

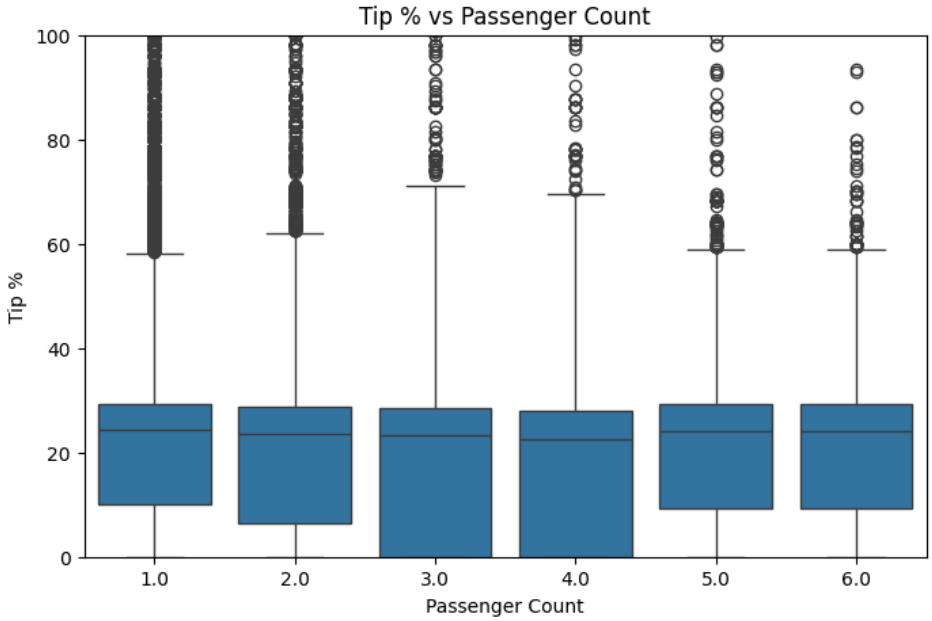


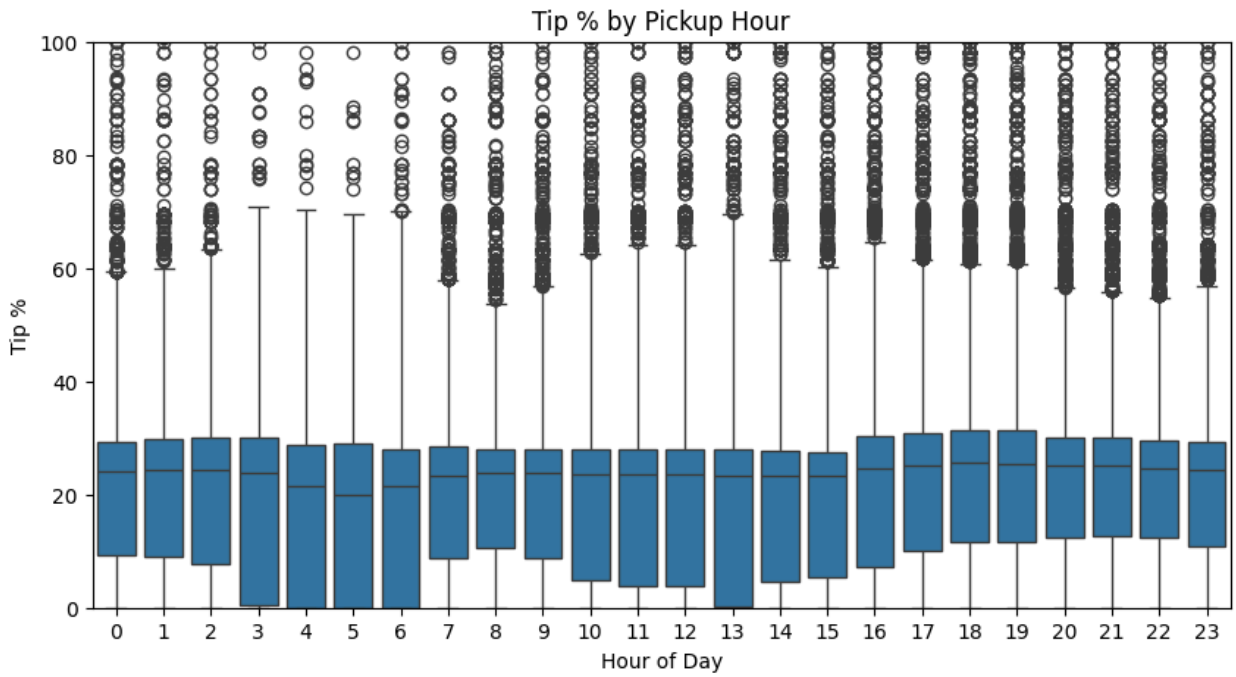
3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion



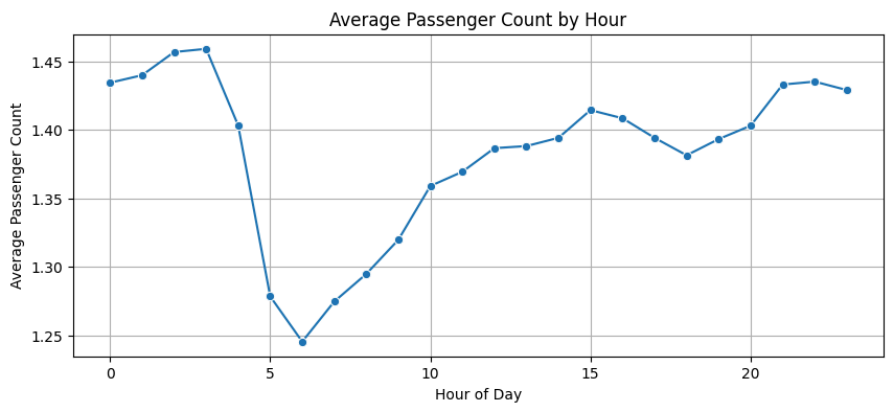
3.2.13. Analyse the tip percentages

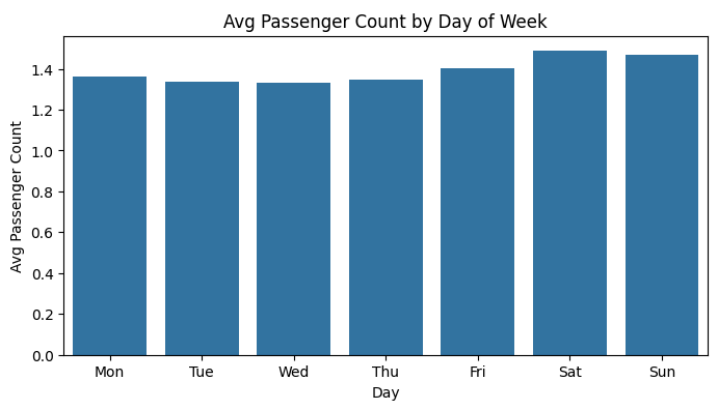






3.2.14. Analyse the trends in passenger count





3.2.15. Analyse the variation of passenger counts across zones

zone_passenger_avg = df.groupby('PULocationID')['passenger_count'].mean().reset_index()

zone_passenger_avg.columns = ['location_id', 'avg_passenger_count']

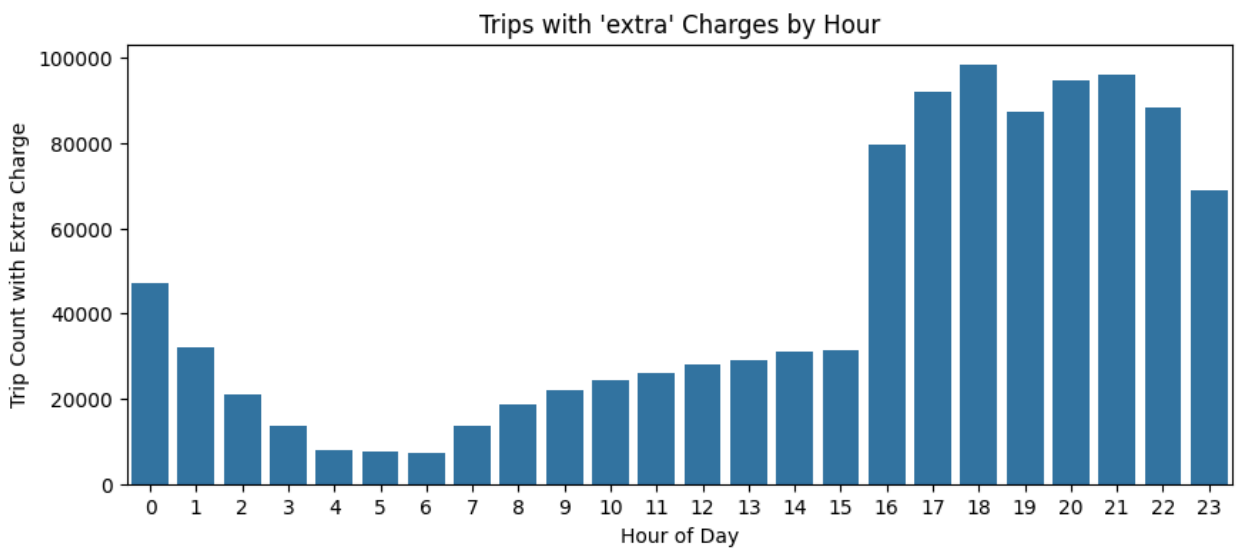
3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

extra: Applied in 62.14% of trips

mta_tax: Applied in 99.85% of trips

improvement_surcharge: Applied in 100.00% of trips

congestion_surcharge: Applied in 93.93% of trips



Top 10 Zones with Most Trips Having Extra Charges:

	PULocationID	extra_trip_count
110	138	58497
131	161	52777
194	237	47767
193	236	39238
132	162	39019
187	230	38694
113	142	36881
151	186	35580
133	163	32585
66	79	31984

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

Busiest Hour: 18 with 121738 trips, Hours with least traffic are 4 and 5.

Top 10 Pickup Zones: PULocationID 237, 161, 236, 132, 162, 186, 138, 142, 230, 170.

Top 10 Dropoff Zones: DOLocationID 236, 237, 161, 230, 170, 162, 142, 239, 141, 68.

By looking at the above data we can see that there are repeated locationID in both pick and drop off zones this lets us come to the conclusion most of the trips are within the zone or the zones hold the most amount of customers, following the above conclusion we need arrange for more taxies in those locationIDs to reduce congestion and reduce the overall fair.

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

Busiest Hour: 18 with 121738 trips,

Busiest Month: May,

Busiest Day: Tuesday.

From the above data we can suggest to provide more incentives for the drives to make more trips during this period, as during this period the average number of trips is very high and having more number of taxies on road will help reduce extra charges and help with flow of traffic, by providing more supply to the times of heavy demand.

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

By looking at the trip frequency vs distance graphs we can tell that the most number of trips fall below 5 miles both vendors provide the same fare per mile in this range to compete in this market we need to provide a fare that is slightly less than the other vendors and charge the average between both vendors for trips above 5 miles, by doing this we can break even any losses that were expected in trips below 5 Miles.